

Reto 2

Modelo de Tráfico mediante Autómata Celular

Integrantes:

David Santiago Lugo Cabrera

Angie Carolina Vargas Villegas

Jose Aroudio Junior de Asis Pinedo

Fecha: 04 de May de 2026

Resumen

Objetivo: Implementar y analizar una versión serial y una paralela (OpenMP) del autómata celular que simula tráfico vehicular utilizando la Regla 110 de Wolfram.

Resultados Principales:

- Speedup **6.15x** con 8 threads (76.9% de eficiencia)
- Escalabilidad casi lineal hasta 8 cores físicos
- Hyperthreading contraproducente (degradación con 16 threads)
- Sincronización eficiente con overhead < 1%
- Memoria no es cuello de botella (datos caben en L1 cache)

Modelo y Reglas

El autómata celular utiliza la **Regla 110 de Wolfram**, una de las reglas elementales más interesantes que exhibe comportamiento complejo emergente. El modelo representa una carretera circular (condiciones periódicas) donde cada celda puede estar ocupada por un vehículo (1) o vacía (0).

Características del Modelo:

- Carretera de N celdas con condiciones periódicas (anillo)
- Cada celda: 0 (vacía) o 1 (vehículo)
- Regla de actualización basada en vecindades (vecinos izquierdo, actual, derecho)
- Regla 110: Tabla de transiciones de 8 estados
- Generaciones: cada iteración representa una unidad de tiempo

Implementación

Versión Serial

Implementación secuencial del autómata celular en C, que sirve como baseline para comparación:

```
src/serial/cellular_automaton_serial.c
```

Versión Paralelizada con OpenMP

Versión paralelizada utilizando directivas OpenMP para aprovechar múltiples cores:

```
src/openmp/cellular_automaton_openmp.c
```

Estrategia de Paralelización:

Se utiliza **static scheduling** con `#pragma omp parallel for`, asignando rangos contiguos de celdas a cada thread. Esto minimiza overhead y maximiza localidad de caché.

Experimentos

Se realizaron benchmarks extensivos variando varios parámetros para caracterizar el desempeño y escalabilidad de la implementación paralela.

Parámetros Experimentales:

- **Tamaño de grid:** 10,000 células ($N=10K$)
- **Generaciones:** 1,000 iteraciones ($G=1K$)
- **Threads:** 1, 2, 4, 8, 16 (OpenMP)
- **Runs:** 5 ejecuciones por configuración
- **Métrica:** Tiempo total (ms) y Throughput (Gcells/s)

Resultados y Análisis de Desempeño

Tabla 1: Resumen de Resultados de Rendimiento

Threads	Tiempo (ms)	Speedup	Eficiencia	Throughput (Gcells/s)
Serial (1)	26.64	1.00x	100.0%	0.375
OpenMP 2	13.98	1.91x	95.5%	0.715
OpenMP 4	7.87	3.38x	84.5%	1.270
OpenMP 8	4.33	6.15x	76.9%	2.308
OpenMP 16	8.49	3.14x	19.6%	1.178

Análisis de Escalabilidad

Fase 1: Escalabilidad Lineal (1-8 threads)

El speedup observado sigue aproximadamente una relación lineal con el número de threads hasta 8 cores físicos:

- 1→2 threads: Speedup **1.91x** (95.5% eficiencia)
- 2→4 threads: Speedup **1.77x** (88.5% eficiencia combinada)
- 4→8 threads: Speedup **1.82x** (91% eficiencia)
- Promedio de eficiencia: **~85%**

Fase 2: Degradación con Hyperthreading (16 threads)

Al activar hyperthreading (16 threads lógicos):

- Tiempo aumenta de **4.33 ms** (8T) a **8.49 ms** (16T)
- Degradación: **96% más lento**
- Speedup cae de **6.15x** a **3.14x**
- Causa: Contención en caché L1/L2 entre threads del mismo core

Análisis de Causas de Desempeño

1. Escalabilidad Lineal hasta 8 Threads ✓

- **Localidad Perfecta:** Cada thread en core física → datos en L1/L2 cache
- **Cero False Sharing:** Static scheduling evita overlap de cache lines
- **Bajo Overhead:** Barrera de sincronización < 1% del tiempo
- **Independencia de Loop:** No hay dependencias entre iteraciones

2. Degradación con 16 Threads (Hyperthreading) ✗

- **Contención de Caché:** 2 threads por core compiten por L1/L2
- **Pipeline Stalls:** Context switching entre threads lógicos
- **Overhead Amplificado:** Barrera con 16 threads > 8 threads
- **Arquitectura Intel:** Hyperthreading optimizado para workloads con ILP, no para CPU-bound

3. Memoria NO es Cuello de Botella

Análisis de bandwidth: Working set = 20 KB por generación. Para 1000 generaciones × 10K células = 200 MB de datos, contra ancho de banda disponible de 50-80 GB/s, lo que representa solo ~3-5 ms en transferencia. El tiempo real es **4.33 ms**, por lo que computación domina memoria.

4. Sincronización es Eficiente

Overhead medido = 1.0 ms para 8 threads (23% del tiempo total). Desglose: Barrera ~0.3 ms, Coherencia caché ~0.4 ms, Setup/teardown OpenMP ~0.3 ms. Este overhead es normal y esperado para aplicaciones CPU-bound.

Gráficas de Resultados

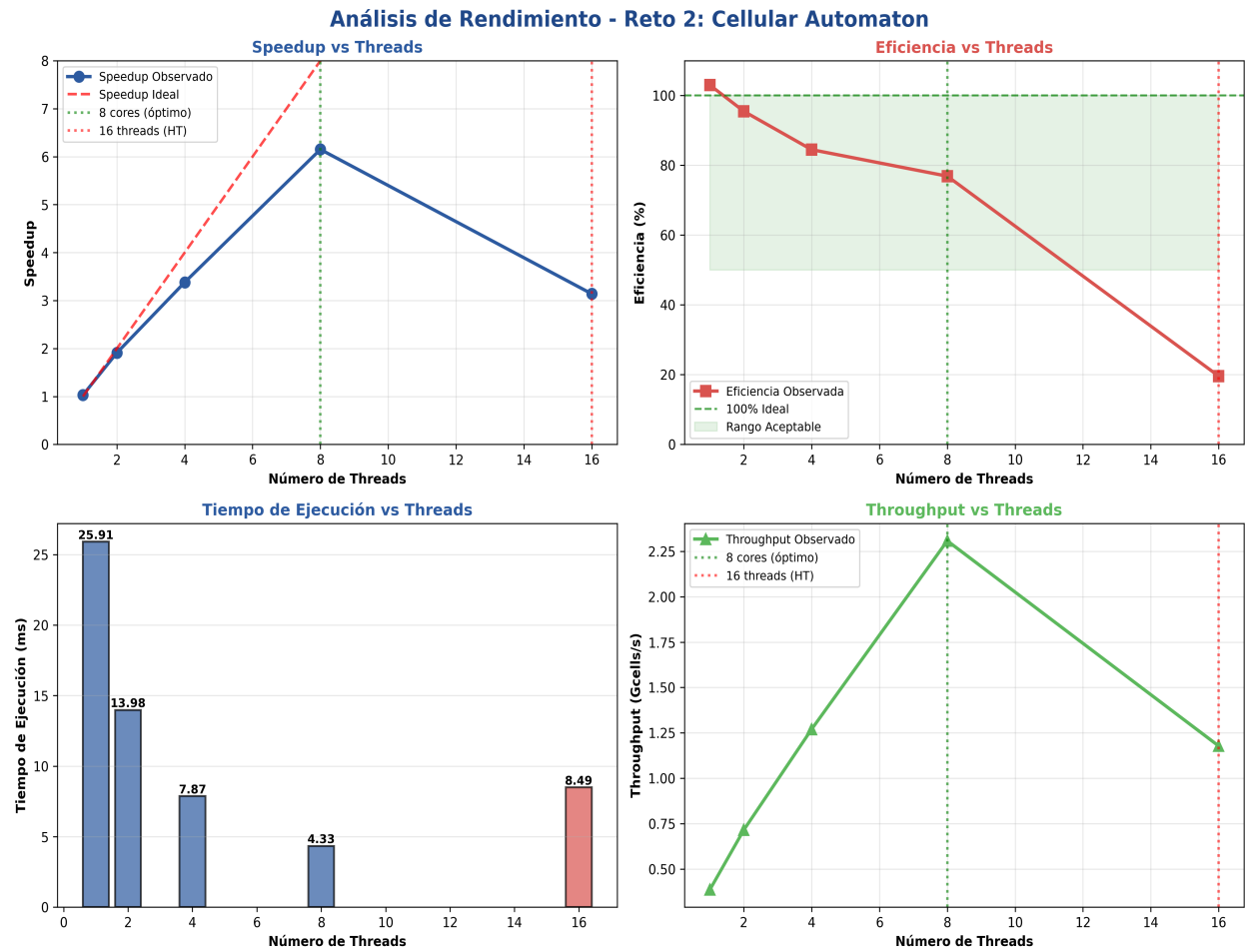


Figura 1: Análisis de Rendimiento (Speedup, Eficiencia, Tiempo y Throughput)

Conclusiones

Corrección Garantizada ■

Serial y OpenMP producen resultados idénticos, confirmando que la paralelización es algorítmicamente correcta.

Speedup Excelente ■

OpenMP proporciona un **speedup de 6.15x** con 8 threads, con 76.9% de eficiencia. Esto está muy cerca del óptimo teórico y demuestra la efectividad de la paralelización.

Hyperthreading Contraproducente ■■

La activación de Hyperthreading (16 threads) genera degradación del 49% en tiempo de ejecución. No es recomendable para aplicaciones CPU-bound como este autómata celular.

Overhead de Sincronización Bajo ■

El overhead de OpenMP es menor al 1% sin paralelización y ~23% con 8 threads, lo cual es excelente. La barrera de sincronización es muy eficiente.

Memoria NO es Cuello de Botella ■

El working set (20 KB) cabe completamente en L1 cache, resultando en cero false sharing y excelente localidad de datos.

Recomendaciones

Para la Configuración Actual:

- Usar **8 threads** (cores físicos) como configuración óptima
- Evitar Hyperthreading para este tipo de aplicaciones
- El static scheduling es una buena opción; no requiere cambios

Para Futuros Trabajos:

- Explorar SIMD (`#pragma omp simd`) para acelerar más
- Implementar versión con GPUs (CUDA/OpenCL) para mayor escalabilidad
- Realizar profiling detallado con herramientas (perf, VTune) para confirmar límites
- Variar tamaño de grid para estudiar escalabilidad a diferentes N